ICS

Association for Computing Machinery

北京航空航天大學
BEIHANG UNIVERSITY

CB-SpMV:A Data Aggregating and Balance Algorithm for Cache-Friendly Block-Based SpMV on GPUs

Xing Cong, Fukai Sun, Yifan Chen, **Chenhao Xie***, Yi Liu, and Depei Qian

School of Computer Science and Engineering

Beihang University, Beijing, China

Salt Lake City, U.S.A. **ICS'25 June 8-11, 2025**

OUTLINE

Background & Motivation

CB-SpMV Methods

Experimental Results

Conclusion

北京航空航天大學
BEIHANG UNIVERSITY

OUTLINE

Background & Motivation

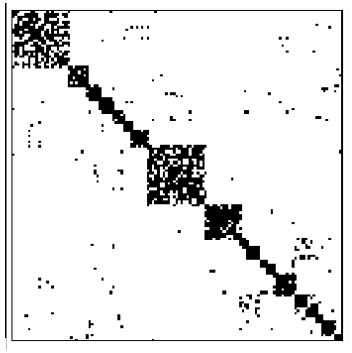
CB-SpMV Methods

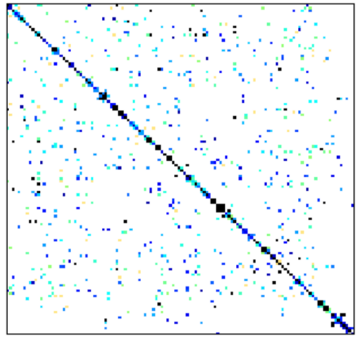
Experimental Results

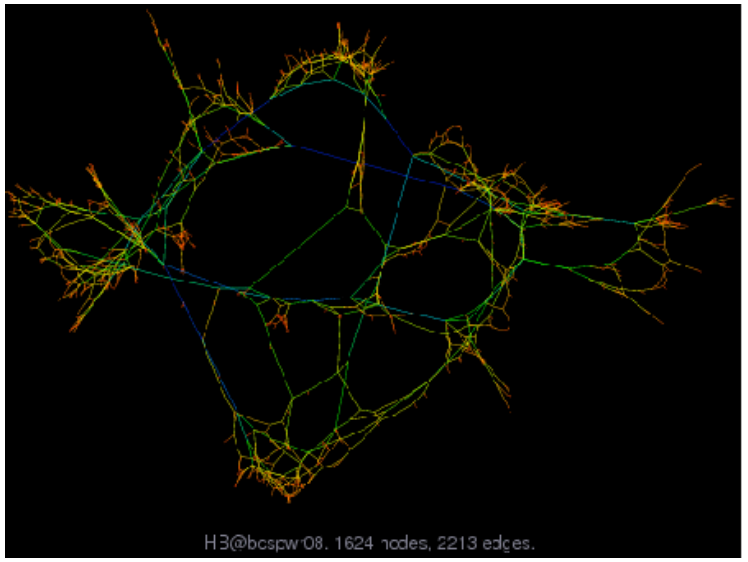
Conclusion

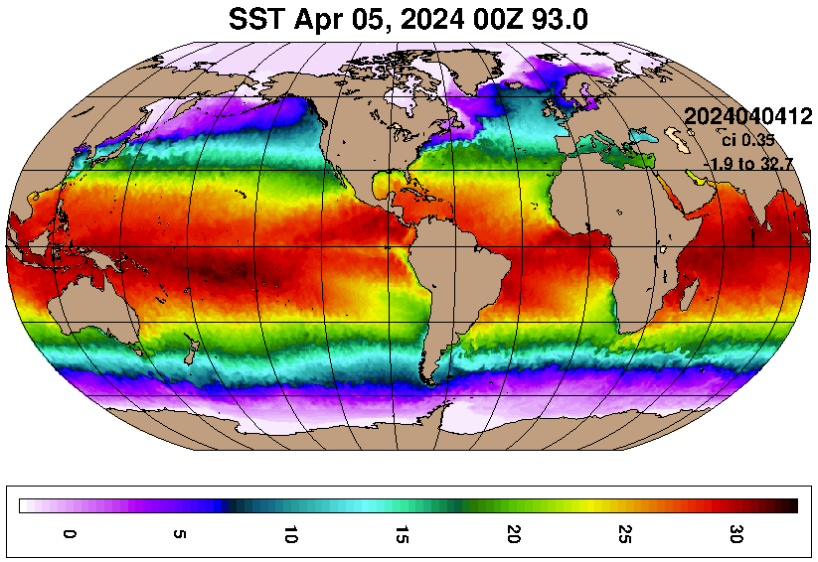
Sparse matrices, where most elements are zero, appear across many domains:

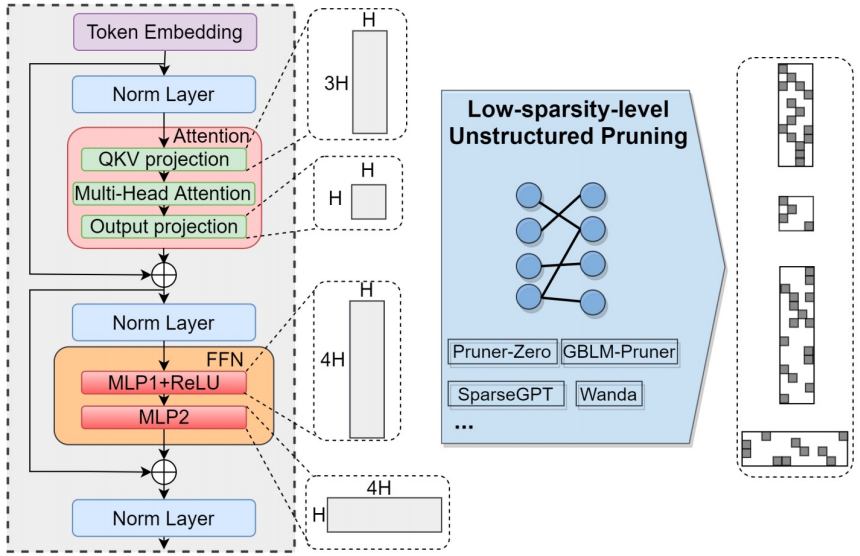
- In **network analysis**, networks are represented as sparse matrices.
- In **scientific computing**, solving large sparse linear systems is common.
- In large-scale models, **sparse attention mechanisms** are becoming a research focus.









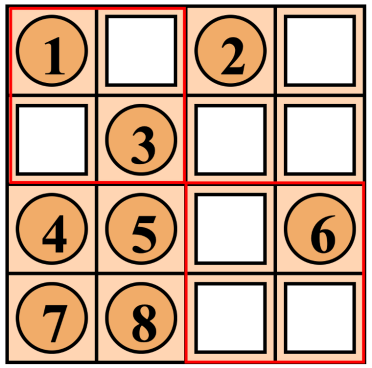


[1] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. ACM Transactions on Mathematical Software 38, 1, Article 1 (December 2011), 25 pages. DOI: https://doi.org/10.1145/2049662.2049663

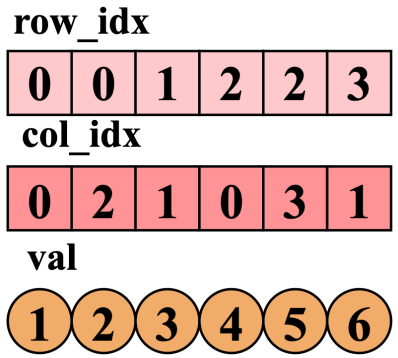
[2] HYCOM Consortium, HYCOM Global 1/12° Analysis (GLBy0.08), https://www.hycom.org

[3] Ruibo Fan, Xiangrui Yu, Peijie Dong, Zeyu Li, Gu Gong, Qiang Wang, Wei Wang, and Xiaowen Chu. 2025. SpInfer: Leveraging Low-Level Sparsity for Efficient Large Language Model Inference on GPUs. In Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25). Association for Computing Machinery, New York, NY, USA, 243–260. https://doi.org/10.1145/3689031.3717481

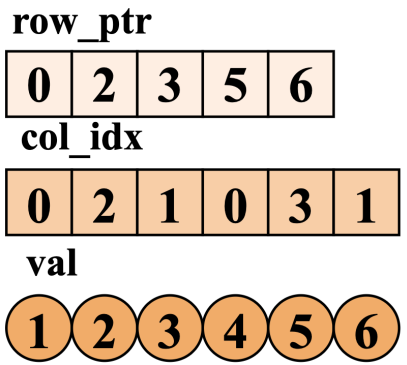
1.2 Sparse Matrix Storage Format



Sparse Matrix

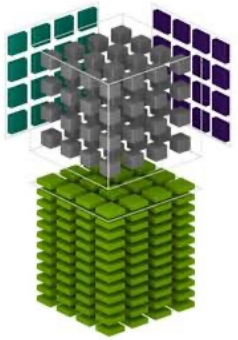


COO Format



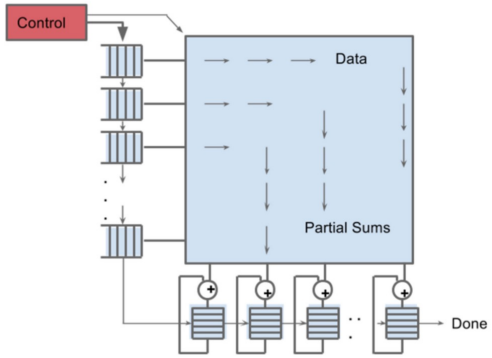
CSR Format

- **COO** is the default format for .mtx files—simple, intuitive, and easy to convert.
- **CSR** is compact and efficient—ideal for row operations and matrix-vector multiplication.



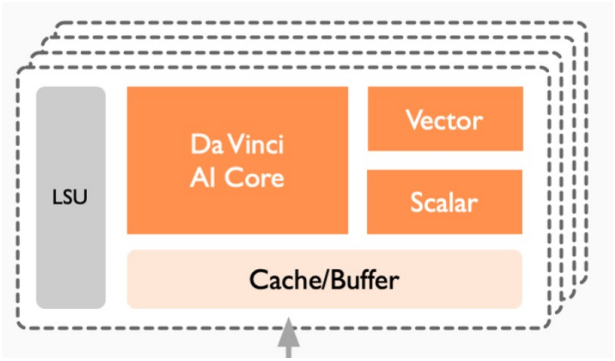
GPU

NVIDIA



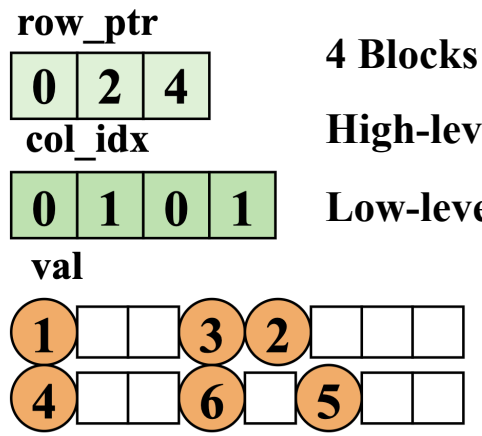
TPU

Google



NPU

HUAWEI

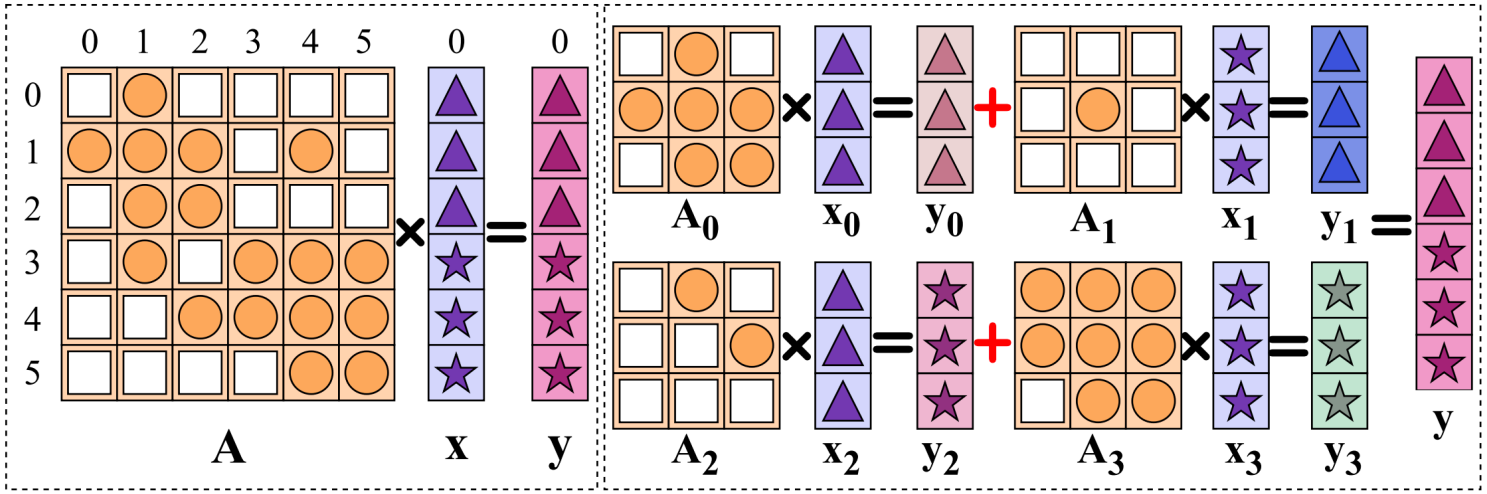


BSR Format

[1] NVIDIA. 2025. Tensor Cores. NVIDIA Corporation. Retrieved May 20, 2025 from https://www.nvidia.com/en-us/data-center/tensor-cores/

[2] Google. 2025. Tensor Processing Units (TPUs). Retrieved May 20, 2025 from https://cloud.google.com/tpu#features

[3] Huawei. 2025. Ascend Documentation Center. Retrieved May 20, 2025 from https://www.hiascend.com/zh/document



(a) General SpMV (b) Blocked SpMV

Algorithm 1 A pseudocode of parallel CSR SpMV.

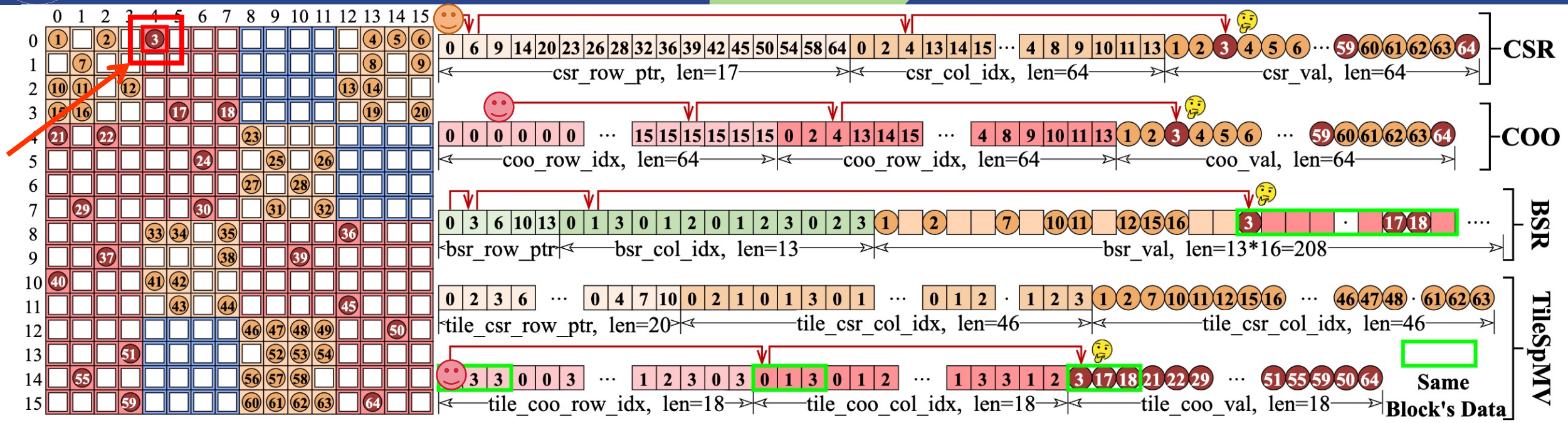
```
1:  s_x ← x                                   ▷ s_x is in shared memory
2:  for i = 0 to m in parallel do
3:      sum ← 0
4:      for j = row_ptr[i] to row_ptr[i + 1] do
5:          //m + 1 elements are spanned from row_ptr to col_idx
6:          val_x ← s_x[col_idx[j]]
7:          //nnz elements are spanned from col_idx to csr_val
8:          sum ← sum + val_x × csr_val[j]
9:      end for
10:     y[i] ← sum
11: end for
```

- Sparse Matrix-Vector Multiplication (**SpMV**) computes y = A*x, where A is sparse and x, y are dense vectors. Each yi is the dot product of the i-th row of A with x.
- Since rows are independent, **SpMV is highly parallelizable**.
- However, current SpMV methods often suffer from poor data locality due to **large memory access strides**.

[1] K. Li, W. Yang and K. Li, "Performance Analysis and Optimization for SpMV on GPU Using Probabilistic Modeling," in IEEE Transactions on Parallel and Distributed Systems, vol. 26, no. 1, pp. 196-205, Jan. 2015, doi: 10.1109/TPDS.2014.2308221. keywords: {Sparse matrices;Silicon;Graphics processing units;Instruction sets;Vectors;Arrays;Performance analysis;GPU;performance modeling;probability mass function;sparse matrix-vector multiplication},

[2] Y. Niu, Z. Lu, M. Dong, Z. Jin, W. Liu and G. Tan, "TileSpMV: A Tiled Algorithm for Sparse Matrix-Vector Multiplication on GPUs," 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Portland, OR, USA, 2021, pp. 68-78, doi: 10.1109/IPDPS49936.2021.00016. keywords: {Distributed processing;Design methodology;Supercomputers;Sparse matrices;Testing;sparse matrix-vector multiplication;tiling;GPU},

1.4 Poor Data Locality



- Different types of arrays require separate GPU memory allocations and data transfers, leading to **large memory access strides** during calculations and poor data locality.
- Unlike CPUs, GPUs have no complex prefetching mechanisms, and **each thread has a very small cache size**, further **reducing cache hit rates**.

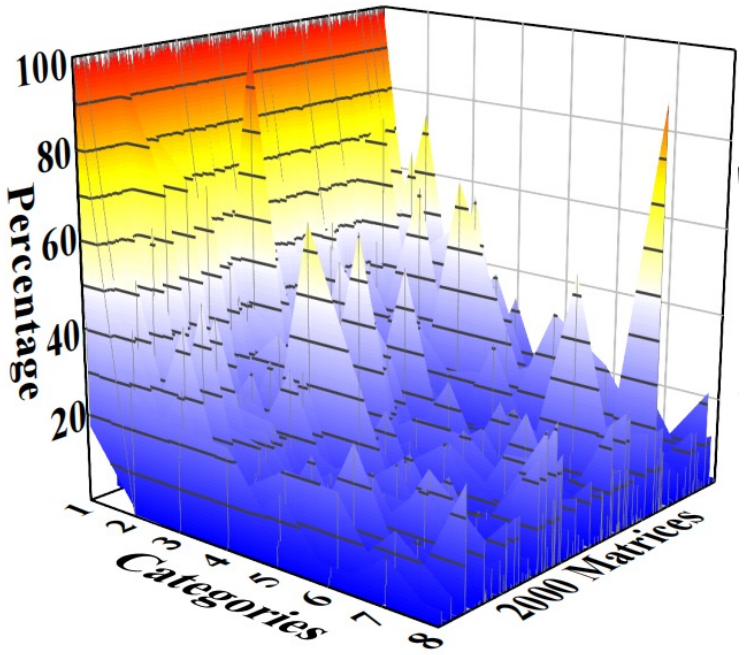
[1] J. L. Greathouse and M. Daga, "Efficient Sparse Matrix-Vector Multiplication on GPUs Using the CSR Storage Format," SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, New Orleans, LA, USA, 2014, pp. 769-780, doi: 10.1109/SC.2014.68.

[2] NVIDIA. 2025. cuSPARSE Library Documentation. Retrieved May 20, 2025 from https://docs.nvidia.com/cuda/cusparse/index.html

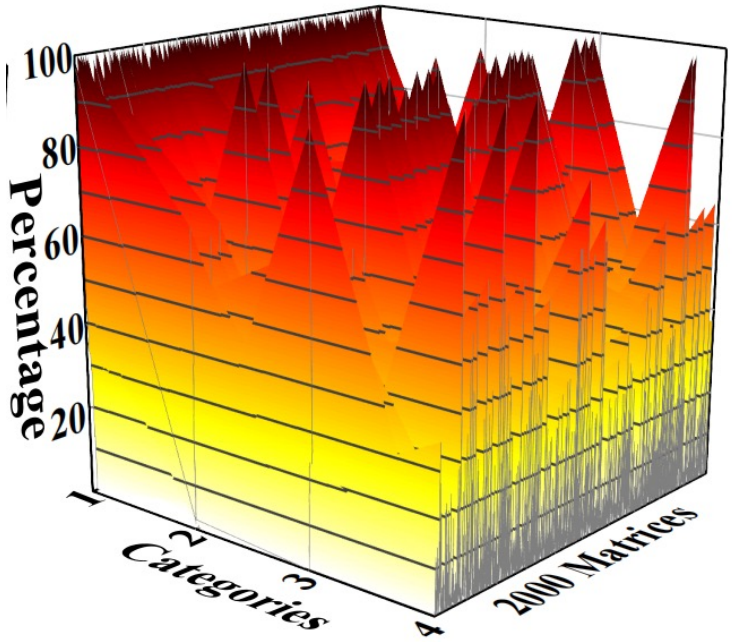
[3] Y. Niu, Z. Lu, M. Dong, Z. Jin, W. Liu and G. Tan, "TileSpMV: A Tiled Algorithm for Sparse Matrix-Vector Multiplication on GPUs," 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Portland, OR, USA, 2021, pp. 68-78, doi: 10.1109/IPDPS49936.2021.00016. keywords: {Distributed processing;Design methodology;Supercomputers;Sparse matrices;Testing;sparse matrix-vector multiplication;tiling;GPU},

1.5 Hardware Under-Utilization

Categories	1	2	3	4	5	6	7	8
(a)	**1-32 (80%↑)**	33-64	65-96	97-128	129-160	161-192	193-224	225-256
(b)	**1-8 (50%↑)**	**9-16 (20%↑)**	17-24	25-32	/	/	/	/



(a) Proportion of Block Categories

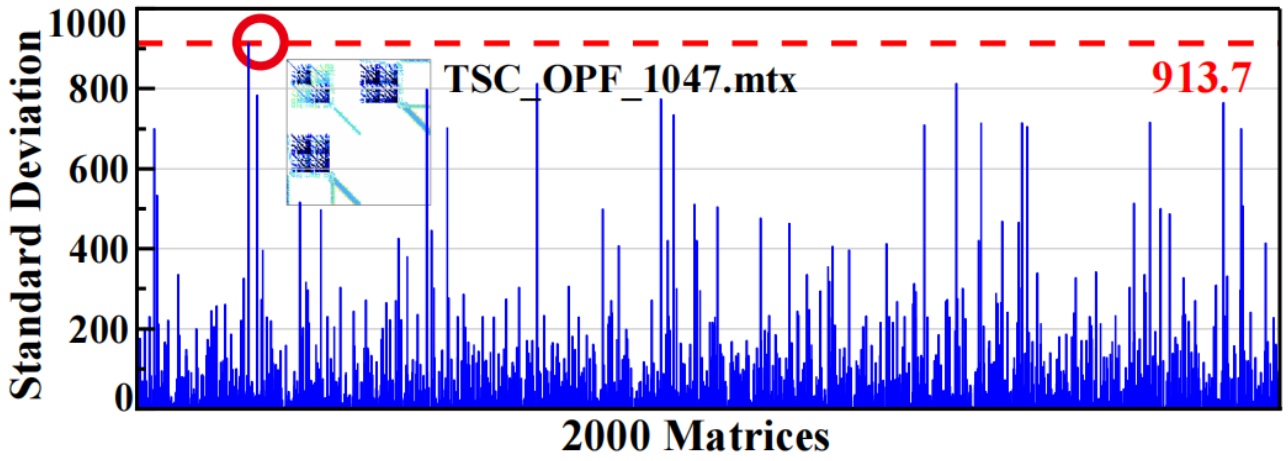


(b) Proportion of 4 Subcategories in 1-32 Block Categories

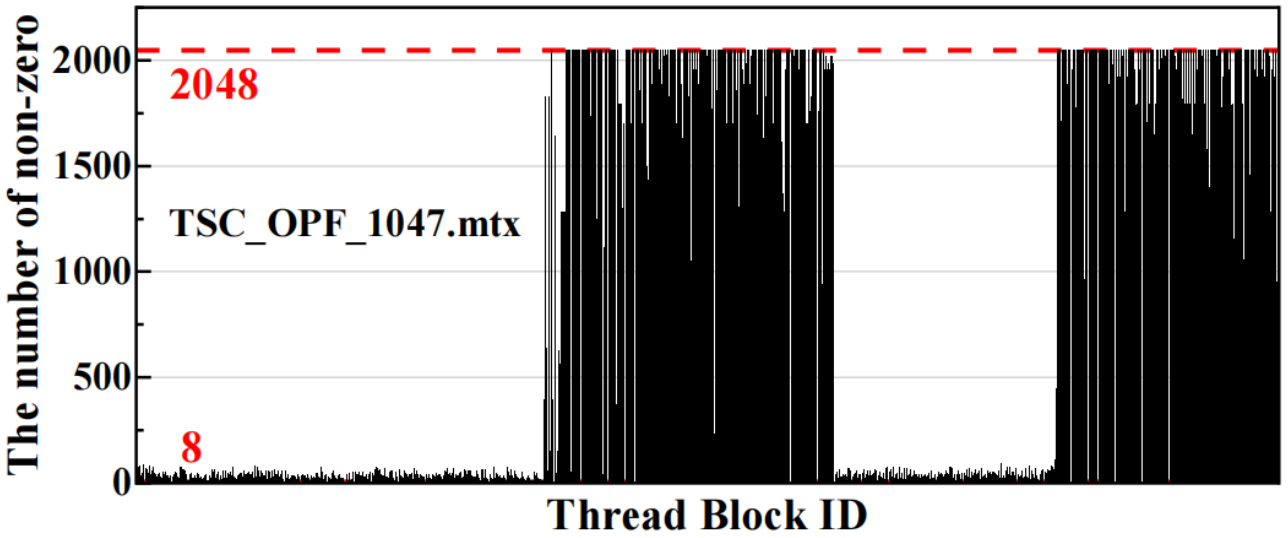
- (a): For 2,000 SuiteSparse matrices 81.89% of sub-blocks have 1–32 nonzeros on average.
- (b): 59.36% of sub-blocks have 1–8 nonzeros (warp utilization <75%). 20.35% of sub-blocks have 9–16 nonzeros (warp utilization <50%).

After the sparse matrix is blocked, most of the sub-blocks are still very sparse!!!

1.6 TB Load Unbanlance



(a) Non-zeros Std Dev per Thread Block for Different Matrix



(b) Thread Block Non-zeros Distribution in the Matrix

- GPUs dispatch thread blocks (multiple warps) to SMs in **round-robin**.
- Sub-blocks with different sparsity cause large variance in non-zeros per thread block, and **execution time depends on the slowest thread block**, severely limiting performance.
- For 2000 matrices, non-zero standard deviation fluctuates greatly; TSC_OPF_1047.mtx reaches 913.7.

The most heavily loaded thread block becomes a performance bottleneck (TB Load Unbanlance), which seriously affects the computational efficiency.

1.7 Challenges in Blocked Matrix

➢ Hardware Constraints

- CSR-style high-level mapping makes direct row-block processing difficult.
- GPU's ***memcpy()*** only supports **single-type data transfer**, limiting simultaneous heterogeneous data operations.
- GPU memory architecture **need alignment** of different data types.

➢ Low Warp Utilization

- Sparse **sub-blocks remain dominant after blocking**; minimum execution unit is 32 threads (one warp), leading to resource waste in underpopulated warps.

➢ Thread Block Load Imbalance

- SM schedules at thread block granularity, **each thread block handles varying numbers of non-zeros**, leading to significant load imbalance across blocks.

OUTLINE

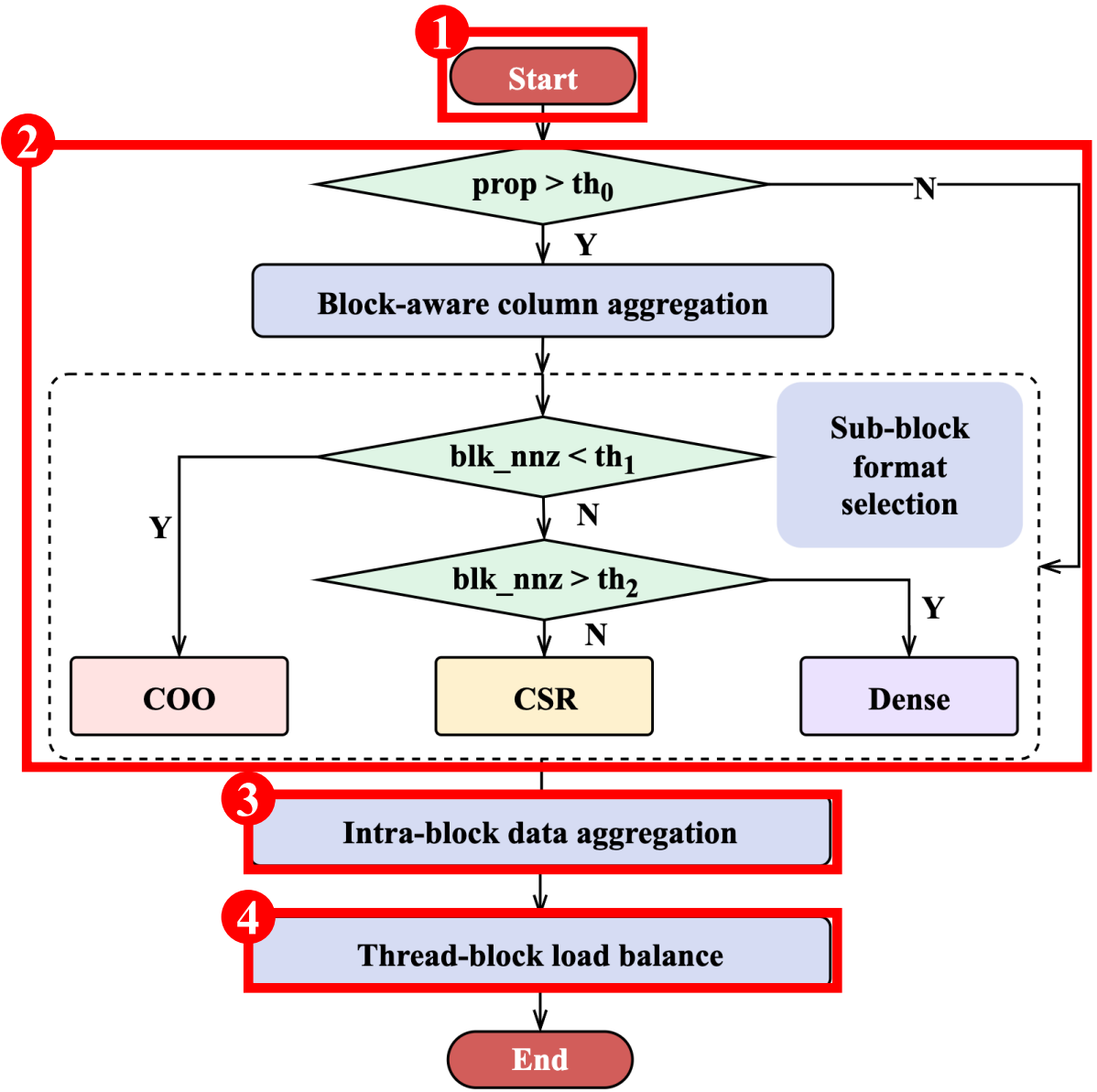
Background & Motivation

CB-SpMV Methods

Experimental Results

Conclusion

2.1 CB-SpMV OverView



➢ 2D Sparse Structure

- High-level: Use COO format for easy mapping
- Low-level: Use hybrid formats to optimize space and speed

➢ Computational Adaptation

- Sparse blocks: Use column aggregation to improve nnz density
- Dense blocks: Use thread-level reductions for acceleration

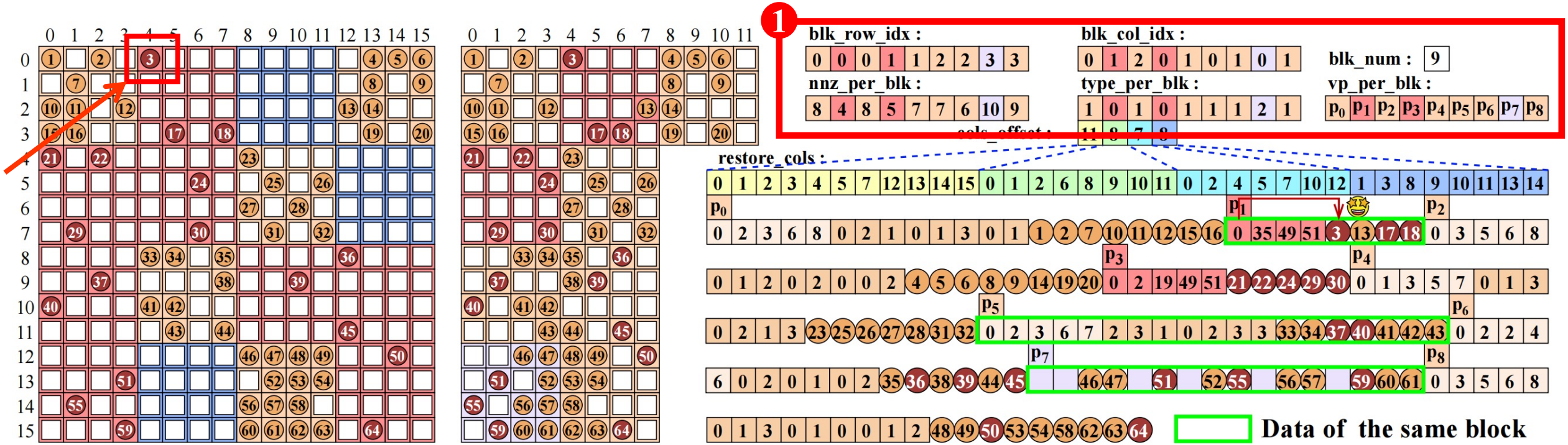
➢ Intra-block Data Aggregation

- Aggregate data within each data-block to ensure contiguous storage in memory

➢ Inter-block Load Balancing

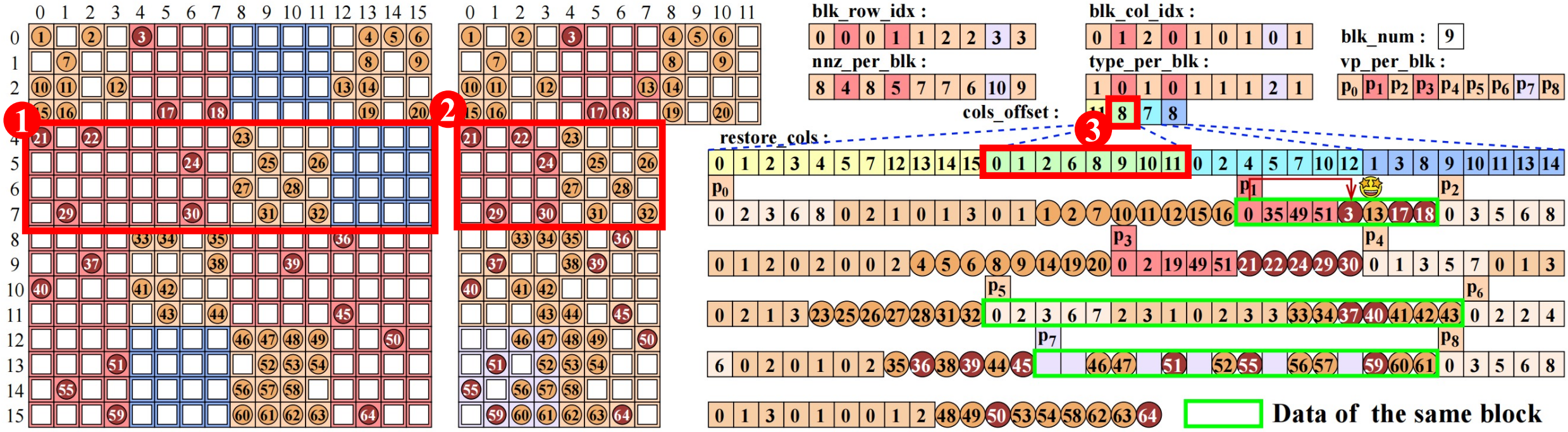
- Reassign data-block pointers per thread-block to balance the number of nonzeros processed by each thread-block

2.2 2D Sparse Structure



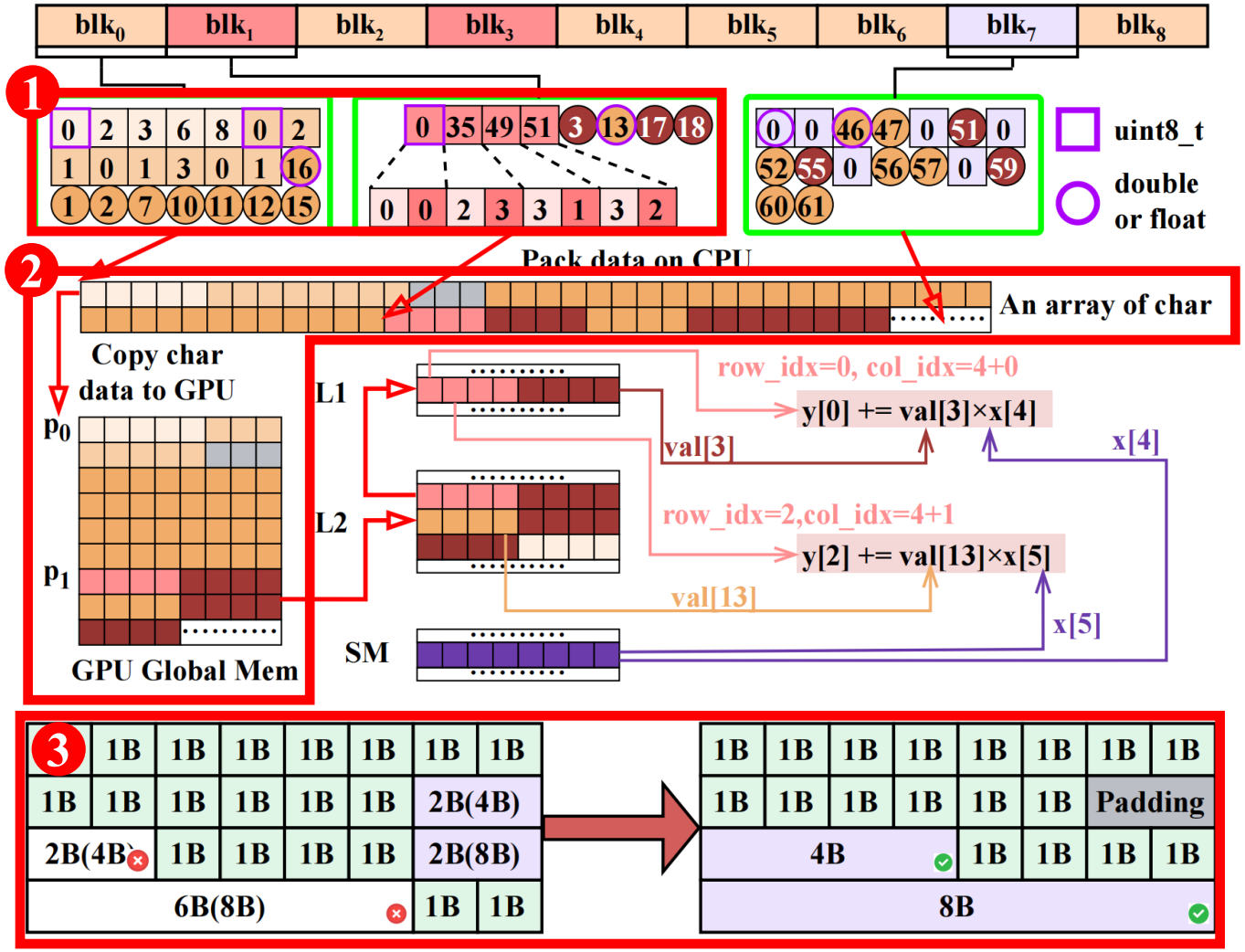
- **High-level**: Use COO format for easy mapping
  - ***blk_row_idx***: Block row index; ***blk_col_idx***: Block column index; ***nnz_per_blk***: Number of nonzeros in each block; ***type_per_blk***: Storage format of each block; ***vp_per_blk***: **GPU memory address of data in each block**
- **Low-level**: Hybrid formats (COO, CSR, Dense) for **faster computation and memory savings**

2.2 Computational Adaptation



- For matrices with sparse blocks >15%: Use **column aggregation**; row block width equals sub-block width
  - ***cols_offset***: Number of nonzero columns in each row block
  - ***restore_cols***: Maps each aggregated column back to the original column index
- For denser blocks: Use **other storage formats** like CSR for space efficiency
  - Leverage thread-level reductions for acceleration

2.3 Intra-block Data Aggregation



➢ Coordinate Compression:

- Reduce indexing overhead
- COO: Combine row/col into a single 8-bit data type
- CSR: Use 8-bit indices for storage

➢ Virtual Pointer Mechanism:

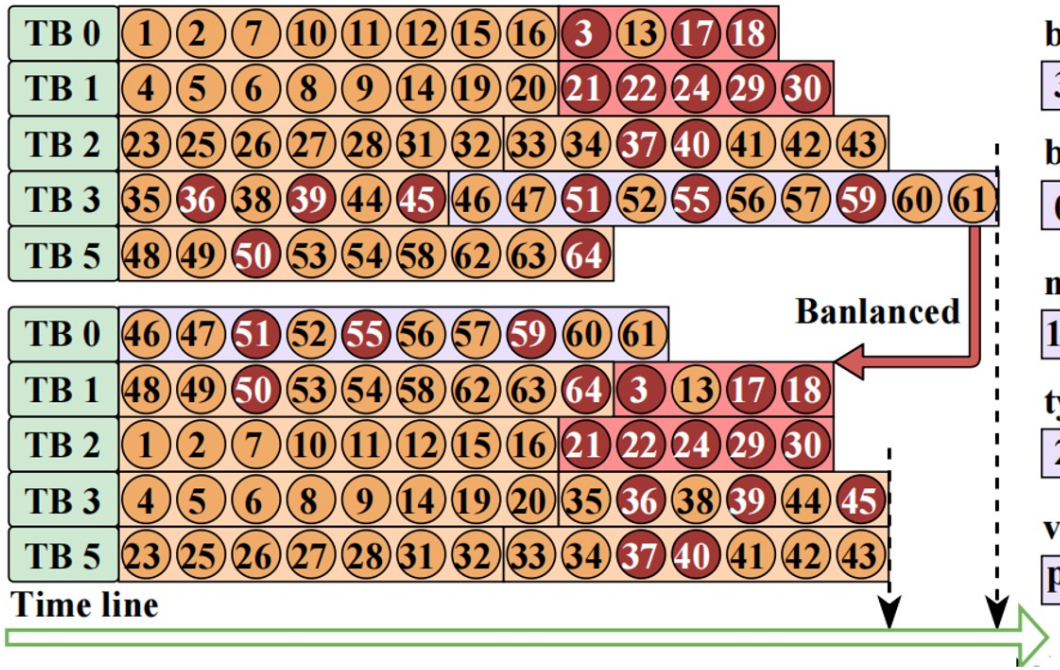
- Store different types/precision blocks in a **unified format**
- Allocate a **contiguous char buffer** on GPU as VP
- Aggregate into a unified sequence on CPU
- Transfer to GPU in a single batch
- Access data via pointer offsets based on VP and block format

➢ Alignment Padding:

- Handle GPU alignment constraints
- Ensure correct memory access and boost performance

Aggregate and store different types of data within the same sub-block contiguously to improve locality, cache hit rate, and reduce memory access overhead.

2.4 Inter-thread-block Load Balance



blk_row_idx :

3	-1	3	0	0	1	0	2	1	2

blk_col_idx :

0	-1	1	1	0	0	2	1	1	0

nnz_per_blk :

10	-1	9	4	8	5	8	6	7	7

type_per_blk :

2	-1	1	0	1	0	1	1	1	1

vp_per_blk :

p_7	/	p_8	p_1	p_0	p_3	p_2	p_6	p_4	p_5

Algorithm 2 A pseudocode of Thread Block Load Balance.

```
1:  //The blk_idx_array contains 3 items: ori,end,nnz.
2:  parallel sort(blk_idx_array, cmp_nnz);
3:  //The pq contains 3 items: loads(min-heap), tb_id, warps.
4:  for i = 0 to blk_num − 1 in parallel do
5:      pqtop ← pq.top(), then pq.pop()
6:      blk_idx_array[i].end ← pqtop.tb_id × 8 + pqtop.warps
7:      pqtop.loads ← pqtop.loads + blk_index_array[i].nnz
8:      pqtop.warps ← pqtop.warps + 1
9:      if pqtop.warps < 8 then
10:         pq.push(pqtop)
11:     end if
12: end for
13: parallel sort(blk_idx_array, cmp_end);
14: for i = 0 to blk_num − 1 in parallel do
15:     vp_per_blk[i] ← vp_per_blk_old[blk_idx_array[i].ori]
16:     //The high-level array (eg.blk_row_idx,...) op similarly
17: end for
```

- Use **priority queue** and **greedy strategy** to assign each data-block to the thread-block with the minimum load, minimizing load imbalance

Thanks to the high-level COO format and VP mechanism, this only requires reordering elements in the high-level array, without changing the data itself

北京航空航天大學
BEIHANG UNIVERSITY

OUTLINE

Background & Motivation

CB-SpMV Methods

Experimental Results

Conclusion

3.1Experimental Setup

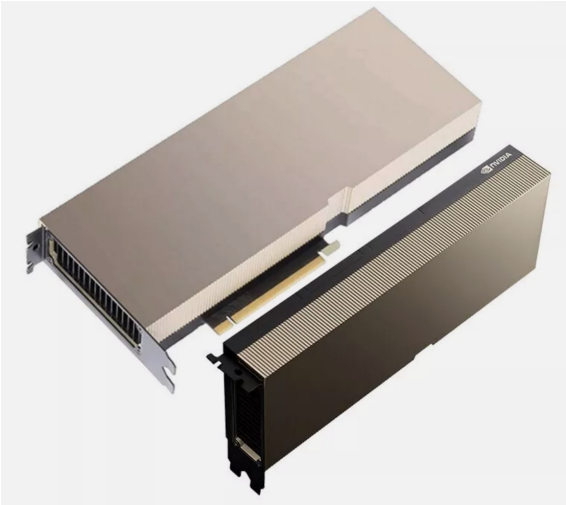


Table 1. The two GPUs and four algorithms evaluated.

Two NVIDIA GPUs	Four Algorithms
(1) NVIDIA A100 (Ampere), 6912 CUDA cores @ 1410 MHz, 40 GB, B/W 1.56 TB/s (2) NVIDIA RTX4090 (Ada Lovelace), 16384 CUDA cores @ 2520 MHz, 24GB, B/W 1.01 TB/s	(1) cuSPARSE [14] (2) TileSpMV [39] (3) DASP [34] **(4) CB-SpMV (this work)**



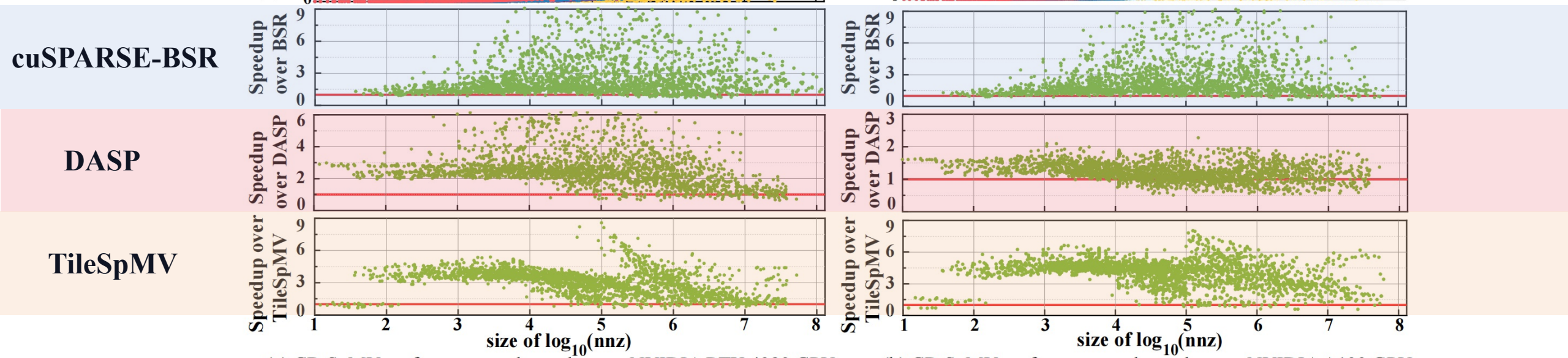
- **cusparse_bsrmv() from cuSPARSE v12.4**: NVIDIA's high-performance SpMV kernel using **BSR format**
- **TileSpMV**: A **block-based advanced SpMV method**, representing state-of-the-art block-level optimization
- **DASP**: A CSR-based SpMV method **utilizing Tensor Cores** for acceleration, showcasing deep hardware-aware optimizations

[1] NVIDIA. 2025. cuSPARSE Library Documentation. Retrieved May 20, 2025 from https://docs.nvidia.com/cuda/cusparse/index.html

[2] Y. Niu, Z. Lu, M. Dong, Z. Jin, W. Liu and G. Tan, "TileSpMV: A Tiled Algorithm for Sparse Matrix-Vector Multiplication on GPUs," 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Portland, OR, USA, 2021, pp. 68-78, doi: 10.1109/IPDPS49936.2021.00016. keywords: {Distributed processing;Design methodology;Supercomputers;Sparse matrices;Testing;sparse matrix-vector multiplication;tiling;GPU},

[3] Yuechen Lu and Weifeng Liu. 2023. DASP: Specific Dense Matrix Multiply-Accumulate Units Accelerated General Sparse Matrix-Vector Multiplication. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23). Association for Computing Machinery, New York, NY, USA, Article 73, 1–14. https://doi.org/10.1145/3581784.3607051

3.2 Performance Comparison

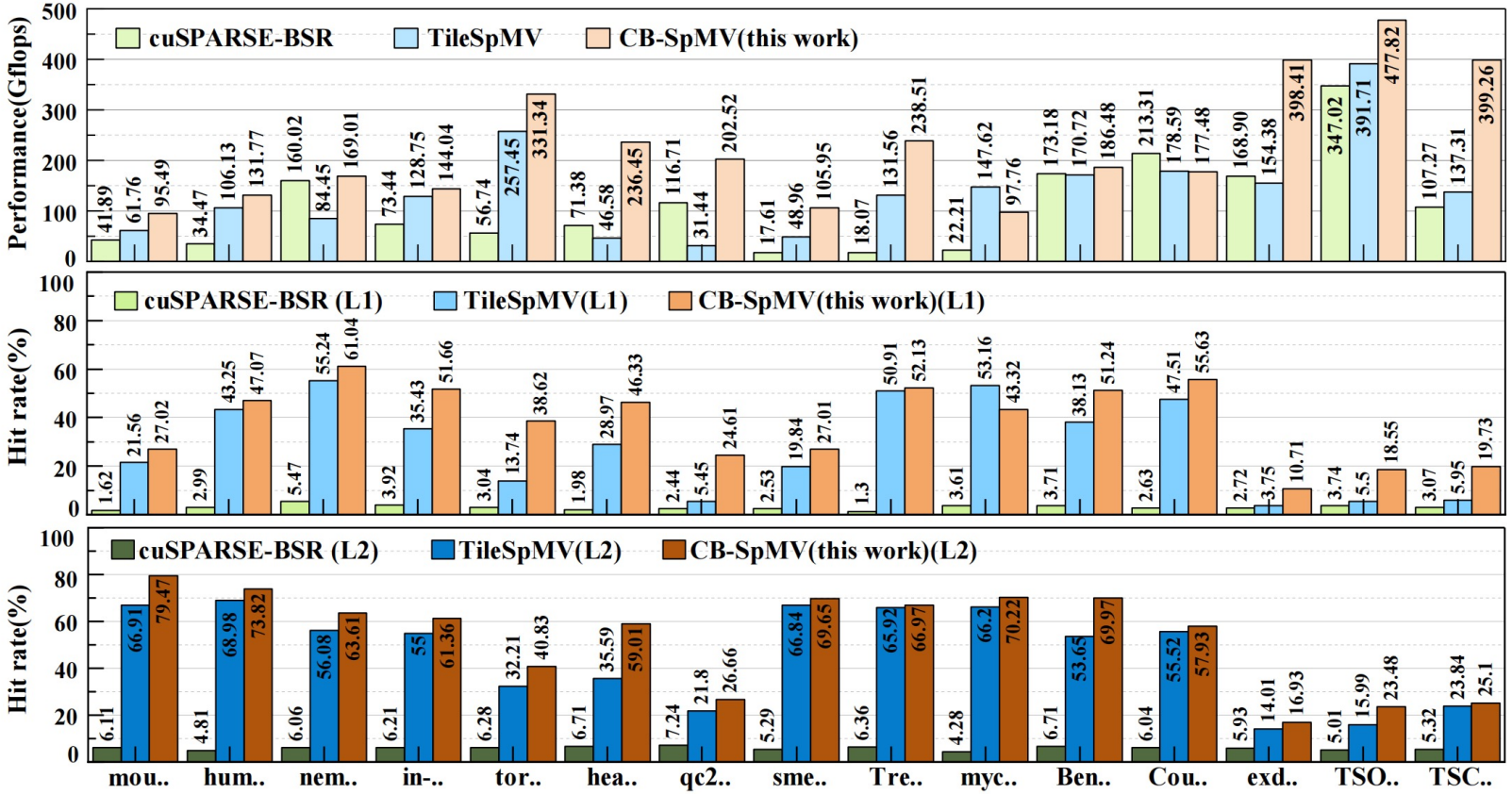


(a) CB-SpMV performance and speedups on NVIDIA RTX 4090 GPU

(b) CB-SpMV performance and speedups on NVIDIA A100 GPU

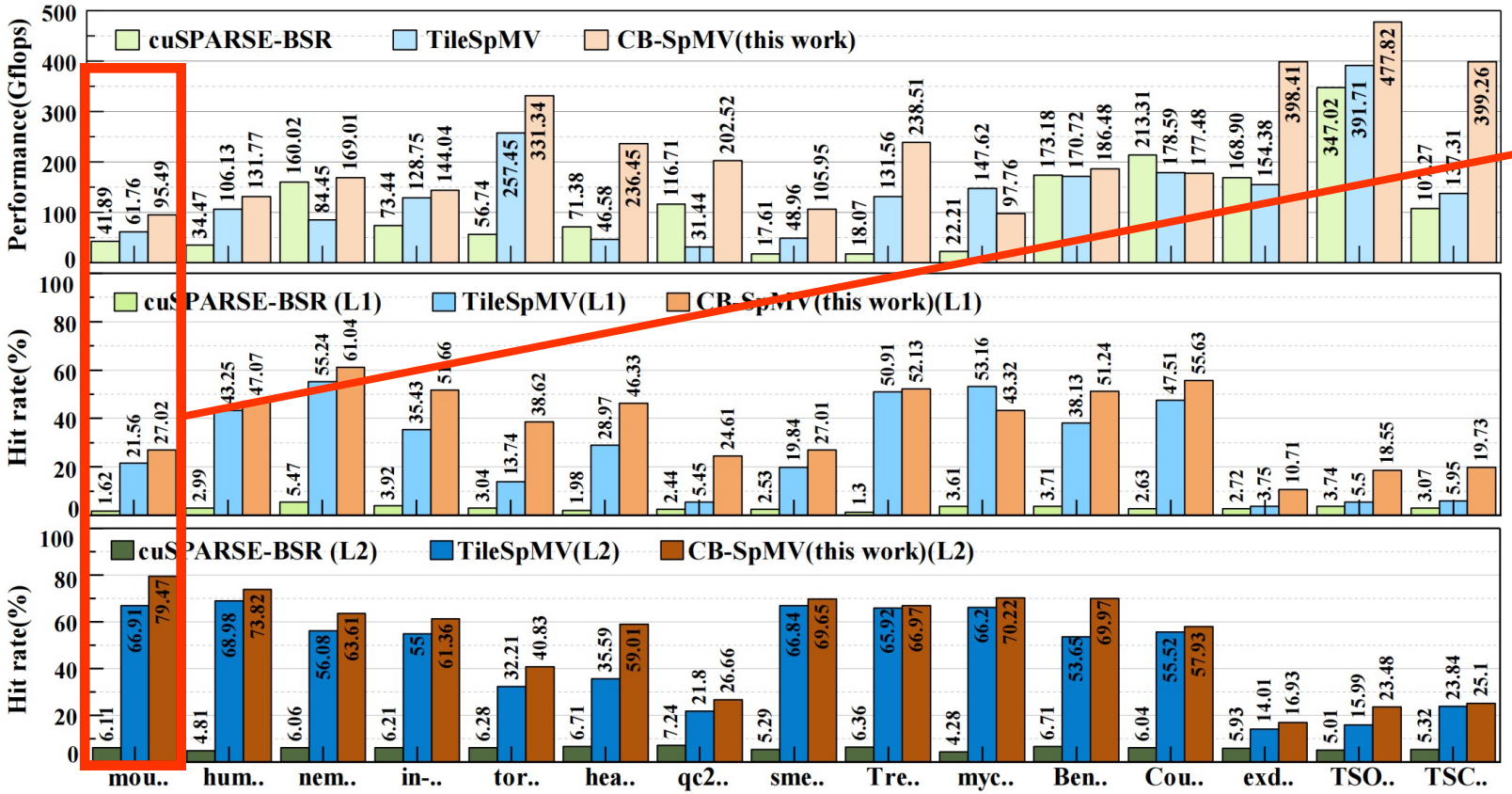
- Compared to **cuSPARSE** (best of four variants): **2.95×** speedup on 4090; **2.99×** speedup on A100
- Compared to **TileSpMV**: **3.06×** speedup on 4090; **3.95×** speedup on A100
- Compared to **DASP**: **2.76×** speedup on 4090; **1.21×** speedup on A100

3.3 Cache hit ratio analysis



- L1 Cache Hit Rate: **35.33%** higher than cuSPARSE; **9.75%** higher than TileSpMV
- L2 Cache Hit Rate: **47.78%** higher than cuSPARSE; **7.10%** higher than TileSpMV

3.3 Cache hit ratio analysis

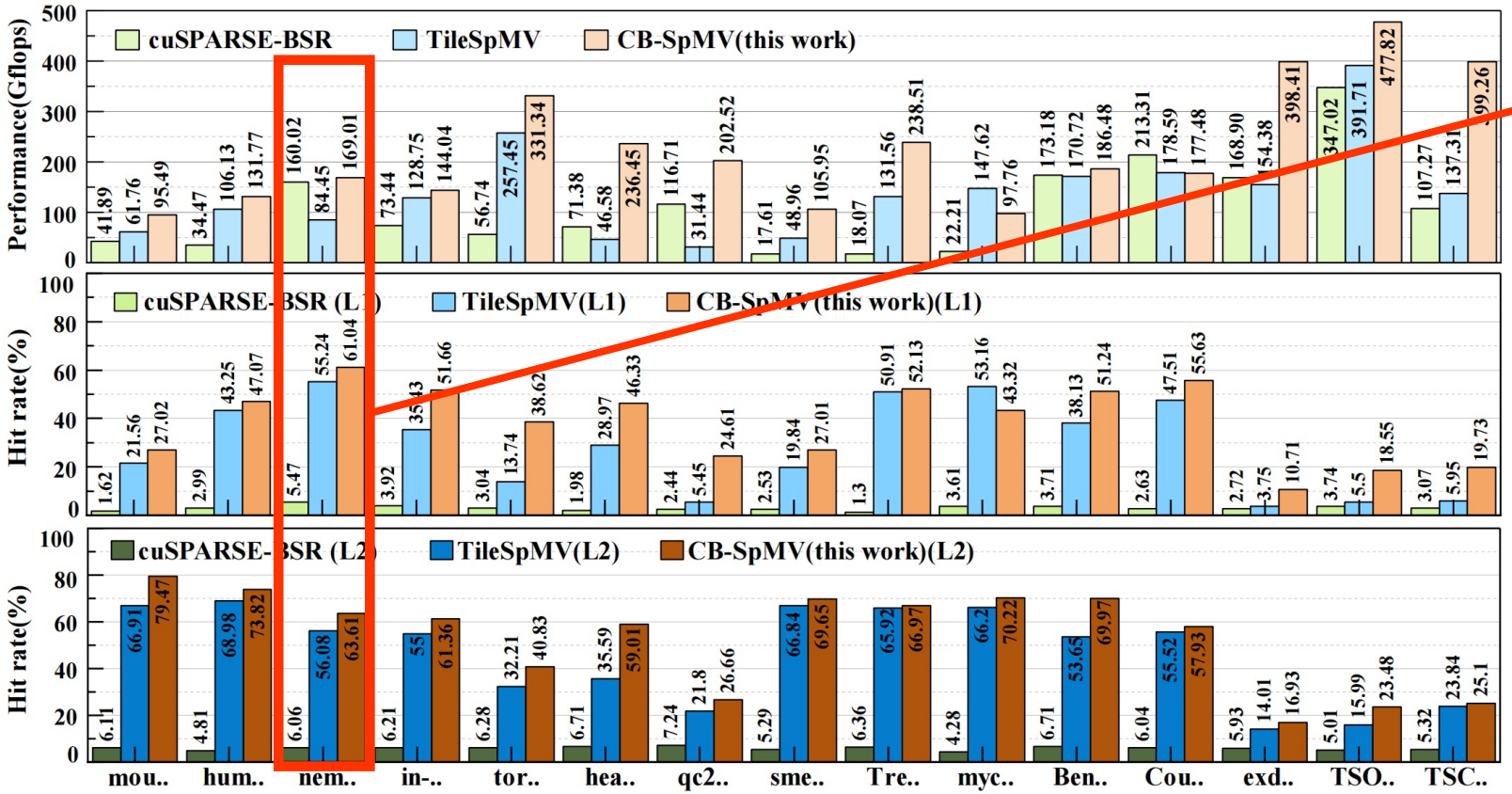




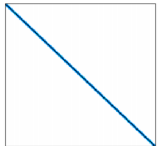
Matrices with more **dense sub-blocks** tend to have higher **L2 Cache** hit rates. After aggregation, dense sub-blocks typically require larger storage, which the **L2 Cache** effectively buffers.

- L1 Cache Hit Rate: **35.33%** higher than cuSPARSE; **9.75%** higher than TileSpMV
- L2 Cache Hit Rate: **47.78%** higher than cuSPARSE; **7.10%** higher than TileSpMV

3.3 Cache hit ratio analysis



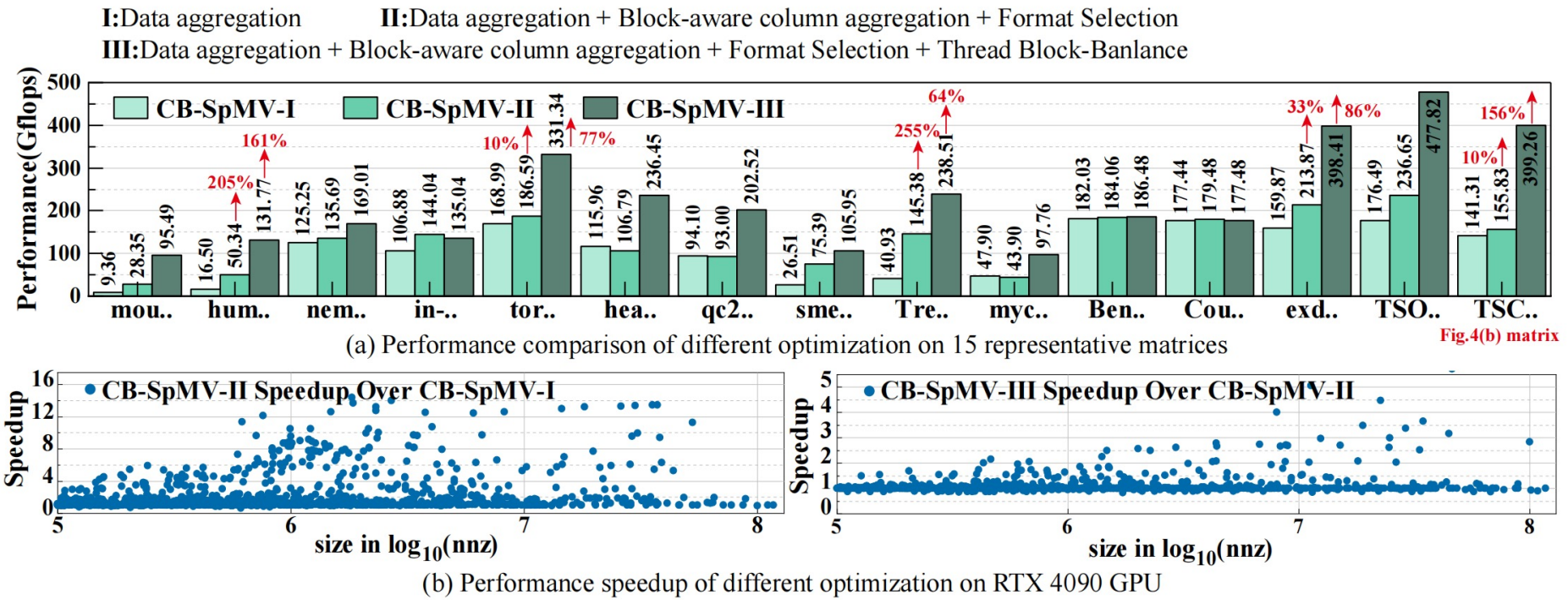
nemeth07



Matrices with more **sparse sub-blocks** tend to have higher **L1 Cache** hit rates. Sparse blocks contain fewer data elements, making it likely that the entire block fits within the **L1 Cache** for a warp.

- L1 Cache Hit Rate: **35.33%** higher than cuSPARSE; **9.75%** higher than TileSpMV
- L2 Cache Hit Rate: **47.78%** higher than cuSPARSE; **7.10%** higher than TileSpMV

3.4 Ablation Experiment



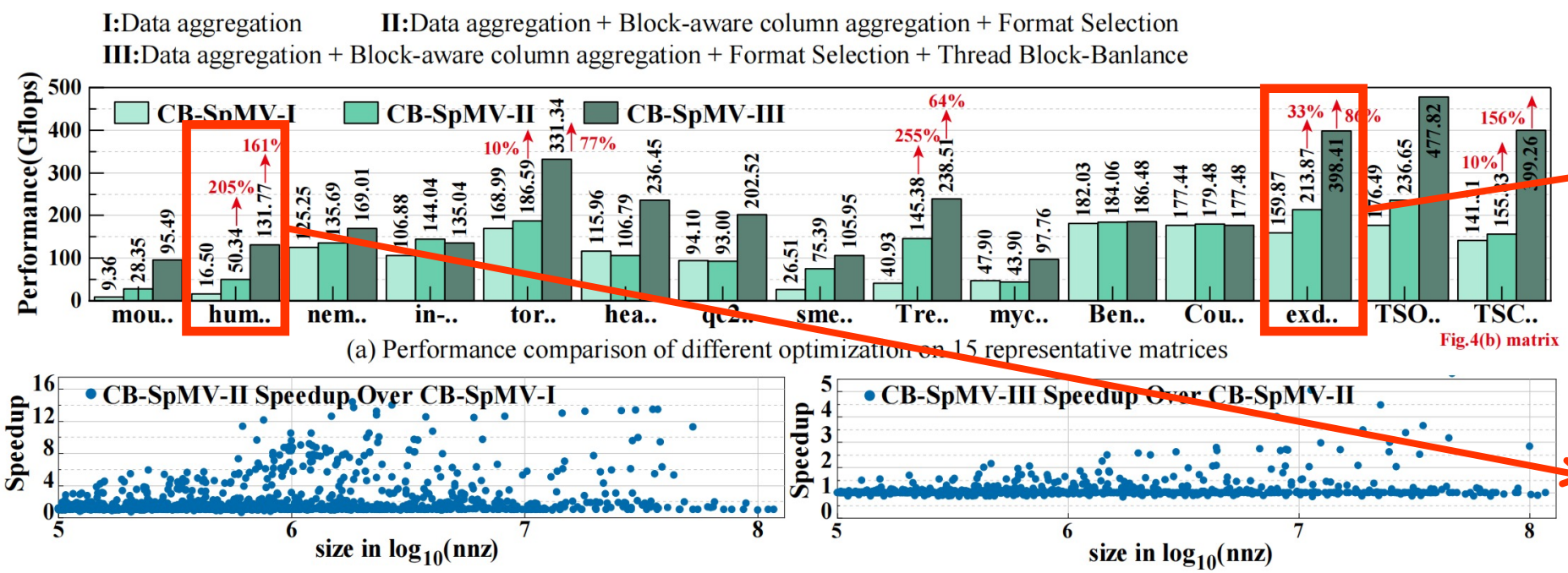
(a) Performance comparison of different optimization on 15 representative matrices

Fig.4(b) matrix

(b) Performance speedup of different optimization on RTX 4090 GPU

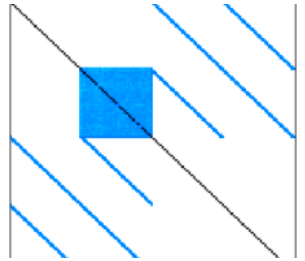
- After data aggregation + column aggregation + format selection (II):
  - Achieves an average **2.22×** speedup
- After applying inter-thread-block load balancing (III):
  - Achieves an average **2.37×** speedup

3.4 Ablation Experiment

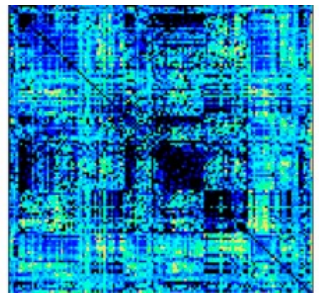


(a) Performance comparison of different optimization on 15 representative matrices

(b) Performance speedup of different optimization on RTX 4090 GPU



exdata_1

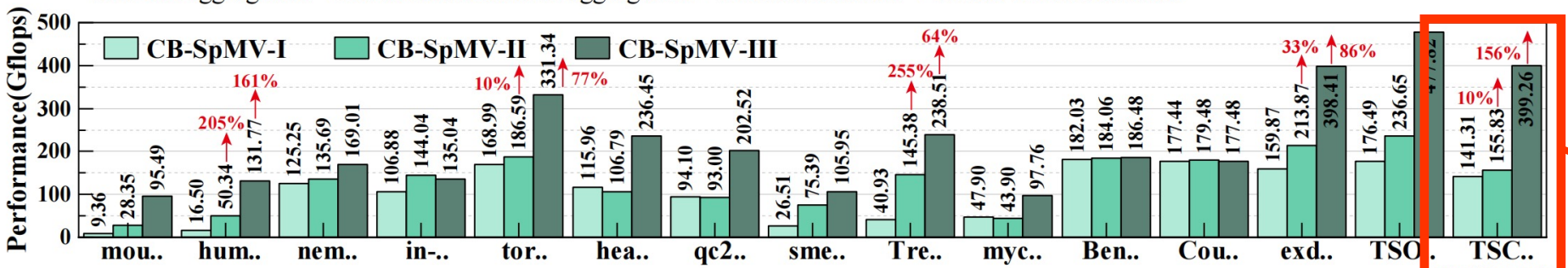


human_gene1

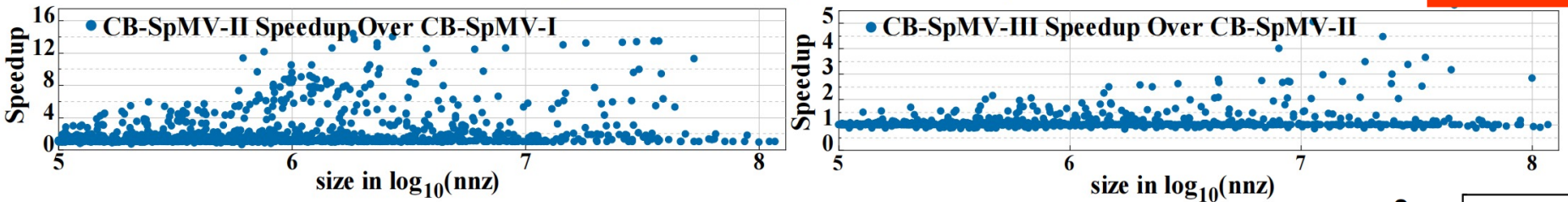
- After data aggregation + column aggregation + format selection (II):
  - Achieves an average **2.22×** speedup
- After applying inter-thread-block load balancing (III):
  - Achieves an average **2.37×** speedup

3.4 Ablation Experiment

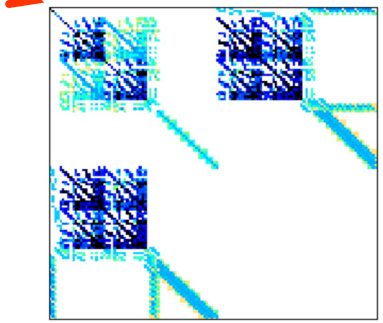
I:Data aggregation **II:**Data aggregation + Block-aware column aggregation + Format Selection
III: Data aggregation + Block-aware column aggregation + Format Selection + Thread Block-Banlance

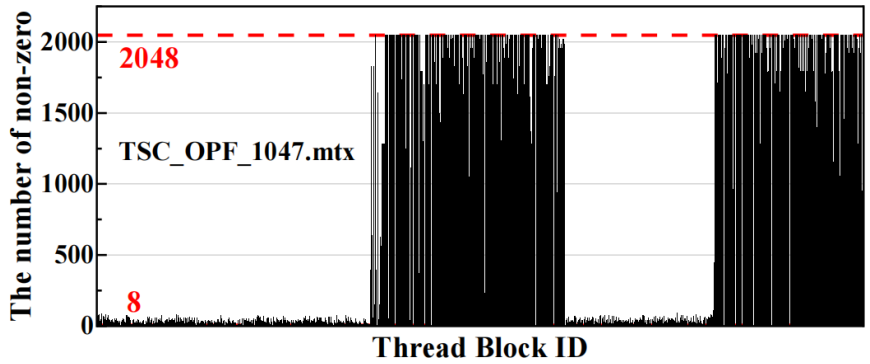


(a) Performance comparison of different optimization on 15 representative matrices



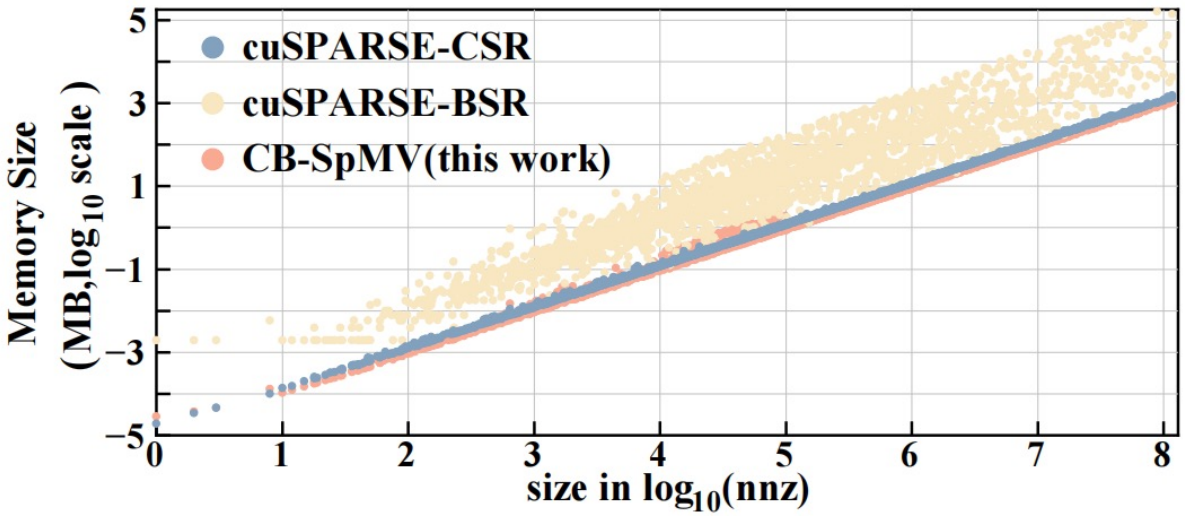
(b) Performance speedup of different optimization on RTX 4090 GPU



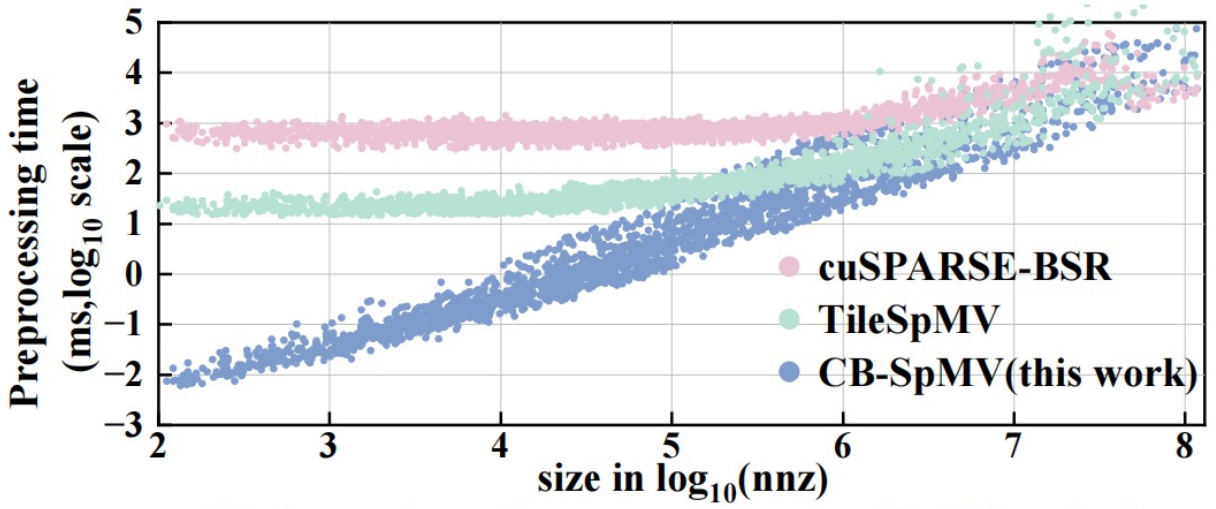


- After data aggregation + column aggregation + format selection (II):
  - Achieves an average **2.22×** speedup
- After applying inter-thread-block load balancing (III):
  - Achieves an average **2.37×** speedup

3.5 Overhead Analysis



(a) Comparison of memroy size of SpMV methods



(b) Comparison of preprocessing costs of SpMV methods

- CSR: $(m+1)\times 4 + nnz\times 4 + nnz\times 8B$
- BSR: $256 \times 8 \times nnz + (blk_m + 1) \times 4 + nnz \times 4B$
- CB-SpMV: $nnzb \times (4+4+4+1+8) + nnz \times (1+8)B$

- The preprocessing overhead is a **small trade-off** for the significant performance improvements achieved during computation.

北京航空航天大學
BEIHANG UNIVERSITY

OUTLINE

Background & Motivation

CB-SpMV Methods

Experimental Results

Conclusion

CB-SpMV: A Block-Structured Optimized SpMV Method

- **2D Sparse Structure:**
  - Matrix divided into uniform, independent sub-blocks;
  - High-level COO format enables efficient thread-level mapping.
- **Virtual Pointer Data Aggregation:**
  - Unified management of diverse data types; optimized memory access patterns.
- **Block-wise Column Aggregation:**
  - Handles block sparsity variations; integrates sparse data to improve hardware utilization.
- **Format-Adaptive Selection:**
  - Chooses suitable storage formats based on block characteristics.
- **Thread Block-Level Load Balancing:**
  - Optimizes thread scheduling and alleviates nonzero distribution imbalance.

ICS

Association for Computing Machinery

北京航空航天大學
BEIHANG UNIVERSITY



Thanks!



Thank you, and have a great day!

Email： congxingcs@163.com

Wechat： CX__CS

Github: https://github.com/BUAAer-xing/CB_Sparse **(will come soon!)**